

上节回顾

- 生产者--消费者问题

- 典型的两私一公问题，公用信号量与私用信号量
- 公用信号量用于资源（变量）互斥，上锁和开锁操作在同个过程中
- 私用信号量用于进程控制，上锁和开锁操作多交叉在不同过程中，实现两个进程的交叉唤醒
- 在wait操作中必须遵循**先私后公**的规则
- 掌握多个生产进程消费进程交替运行的推导过程

- 哲学家就餐问题

- 记录型信号量会发生死锁
- AND信号量的应用

2.6 进程通信

■ 进程的通信：进程之间的信息交换

- 交换少信息量是低级通信，例如信号量机制
- 交换大信息量是高级通信

■ 信号量机制实现低级进程通信的缺点

- 效率低，通信的数据量太少
- 不透明，用户需要自己编写交换过程的代码，设置数据结构，控制进程互斥与同步等
- 操作系统只是提供内存空间

■ 高级进程通信：定义 **P65**

- 用户直接利用操作系统提供的通信指令
- 高效传送大量数据

2.6 进程通信

2.6.1 高级进程通信类型

- 共享存储器，**Share Memory**，信息无格式
- 消息传递，**Message Passing**，信息有格式
- 管道通信，**Pipe**，以文件方式

2.6 进程通信

1. 共享存储器

- 基于共享数据结构的通信方式

- 生产者-消费者问题中的缓冲池

- 该方式增加程序员负担，**OS**只须提供共享空间，低效，只能传递少量数据

2.6 进程通信

2. 消息传递系统

- 不论是单机系统、多机系统，还是计算机网络，消息传递机制都是用得最广泛的一种进程间通信的机制
- 在消息传递系统中，进程间的数据交换，是以格式化的消息(**message**)为单位
 - 在计算机网络中，又把**message**称为报文
- 程序员直接利用系统提供的一组通信命令(原语)进行通信
- 操作系统隐藏了通信的实现细节，大大减化了通信程序编制的复杂性，而获得广泛的应用。
- 分成直接通信方式和间接通信方式两种

2.6 进程通信

3. 管道通信

- 所谓“管道”，是指用于连接一个读进程和一个写进程以实现他们之间通信的一个共享文件，又名**pipe**文件。
- 写进程：以字符流形式将大量的数据送入共享文件，即送入管道
- 读进程：从管道中接收数据，即从共享文件中读数据
- 实质采用文件的读写操作，支持大数据量传输
- **Unix**的管道实例演示

2.6 进程通信

■ 管道特点

- ❑ 管道只能由使用者进程创建
- ❑ 只有创建进程的同族进程（即自己和子孙进程）才能使用管道
- ❑ 每次写入或读出的信息长度是可变的
- ❑ 因为数据没有来源标志，所以每次数据交换是单向，即每次管道只有一个进程在读取数据，同时只有一个进程在写入数据

■ 管道的协调机制

- ❑ 互斥：读写不能同时使用管道
- ❑ 同步：写完才读，读完才写
- ❑ 确认存在：三方握手

2.6 进程通信

2.6.2 消息传递通信的实现方法

- 直接通信方式：发送进程利用**OS**所提供的发送命令，直接把消息发送给目标进程

- 要求发送进程和接收进程显式地提供对方标识符
- 系统提供下述两条通信命令(原语)

Send(Receiver, message); 发送一个消息给接收进程;

Receive(Sender, message); 接收**Sender**发来的消息; •

例如

原语**Send(P2, m1)**: 表示将消息**m1**发送给接收进程**P2**

原语**Receive(P1, m1)**: 表示接收由**P1**发来的消息**m1**

2.6 进程通信

- 多来源接收：接收进程可与多个发送进程通信，无需事先指定发送进程
 - 例如用于提供打印服务的进程，它可以接收来自任何一个进程的“打印请求”消息。
 - 在接收进程接收消息的原语中的源进程参数，是完成通信后的返回值，接收原语可表示为：

Receive (id, message)

- 即接收进程无法提前确定发送进程，只能在与发送进程通信后才能确定发送进程ID

2.6 进程通信

■ 直接通信原语解决生产者-消费者问题

- 生产者生产一个产品(消息)后, 使用**Send**原语将消息发送给消费者
- 消费者进程使用**Receive**原语来得到一个消息
- 如果消息尚未生产出来, 消费者必须等待, 直至生产者进程将消息发送过来。

repeat

...

produce an item in nextp;

send(consumer, nextp);

until false;

■ 进程的阻塞与唤醒由消息机制内部实现

repeat

receive(producer, nextc);

...

consume the item in nextc;

until false;

2.6 进程通信

■ 间接通信方式:

- ❑ 需要某种共享数据结构的实体作为中介
- ❑ 中介暂存发送进程的消息
- ❑ 接收进程从中介取出发送给自己的消息
- ❑ 中介实体一般称为信箱 (**mailbox**)
- ❑ 间接通信又称为信箱通信
- ❑ 信箱可以提供保存信息、核查用户等功能

2.6 进程通信

间接通信的原语

- 信箱的创建: **Create (mailbox)**
 - 创建包括信箱名字、信箱属性(公用、私用或共享)
 - 共享信箱 还应给出共享者的名字。
- 信箱的撤消: **Delete (mailbox)**
 - 进程不再需要信箱时, 可用信箱撤消原语将之撤消
- 消息的发送: **Send(mailbox, message)**
 - 将一个消息发送到指定信箱
- 消息的接收: **Receive(mailbox, message);**
 - 从指定信箱中接收一个消息

2.6 进程通信

- 信箱类型：私用信箱、公用信箱、共享信箱
 - 信箱创建者也是信箱的拥有者
- 私用信箱：
 - 用户进程创建，随进程结束而消失
 - 单向通信，其他用户进程发送信息，信箱拥有者读信息
- 公用信箱：
 - 操作系统创建，在系统运行期间始终存在。
 - 允许系统中的所有核准进程使用。
 - 双向通信，核准进程从信箱中发送或读取给自己消息
- 共享信箱：
 - 用户进程创建，指明属于共享类型，同时确定共享名单
 - 双向通信，信箱的拥有者和共享者都可以从信箱中发送或读取给自己消息

2.6 进程通信

- 使用信息通信的发送进程和接收进程关系（自学）
- 一对一关系：一个发送进程，一个接收进程
 - 建立一条两者专用的通信链路，不受其他进程干扰
- 多对一关系：多个发送进程，一个接收进程
 - 多个用户进程发送服务请求，一个提供服务器进程提供服务；称为客户/服务器交互(**client/server interaction**)
- 一对多关系：一个发送进程，多个接收进程
 - 发送进程采用广播方式向多个接收者发送消息
- 多对多关系。多个发送进程，多个接收进程
 - 建立一个公用信箱，让多个进程都能向信箱中投递消息；也可从信箱中取走属于自己的消息。

2.6 进程通信

2.6.3 消息传递系统实现中的若干问题（自学）

■ 通信链路(communication link)

- 显式建立：用命令或原语请求系统建立通信链路；使用完后，也用命令或原语拆除。主要用于计算机网络
- 自动建立：无须明确提出建立链路的请求，发送进程只须利用系统提供的发送命令(原语)，系统会自动地建立一条链路。主要用于单机系统

■ 链路类型

- 点一点连接：一条链路只连接两个结点(进程)
- 多点连接：一条链路连接多个($n > 2$)结点(进程)
- 单向链路、双向链路

2.6 进程通信

■ 消息的格式（自学）

- 定长格式：比较短，减少了对消息的处理和存储开销
- 可变长格式：系统付出更多的开销，但方便了用户
- 两种格式各有优缺点，很多系统都同时使用

■ 同步方式（自学）

- 发送进程阻塞、接收进程阻塞
 - 两者间无缓冲，有消息就立刻传递，无消息则同时阻塞
- 发送进程不阻塞、接收进程阻塞
 - 接受进程在有消息才被唤醒，常用于C/S模式
- 发送进程和接收进程均不阻塞
 - 两者间联系着消息队列，只有队列满或队列空才发生阻塞

2.6 进程通信

2.6.4 消息缓冲队列通信机制

■ 数据结构

□ 消息缓冲区

type message buffer=record

sender; 发送者进程标识符 ·

size; 消息长度 ·

text; 消息正文 ·

next; 指向下一个消息缓冲区的指针 ·

end

□ PCB中控制通信的数据项

type processcontrol block=record

...

mq; 消息队列队首指针 ·

mutex; 消息队列互斥信号量 ·

sm; 消息队列资源信号量 ·

...

end

2.6 进程通信

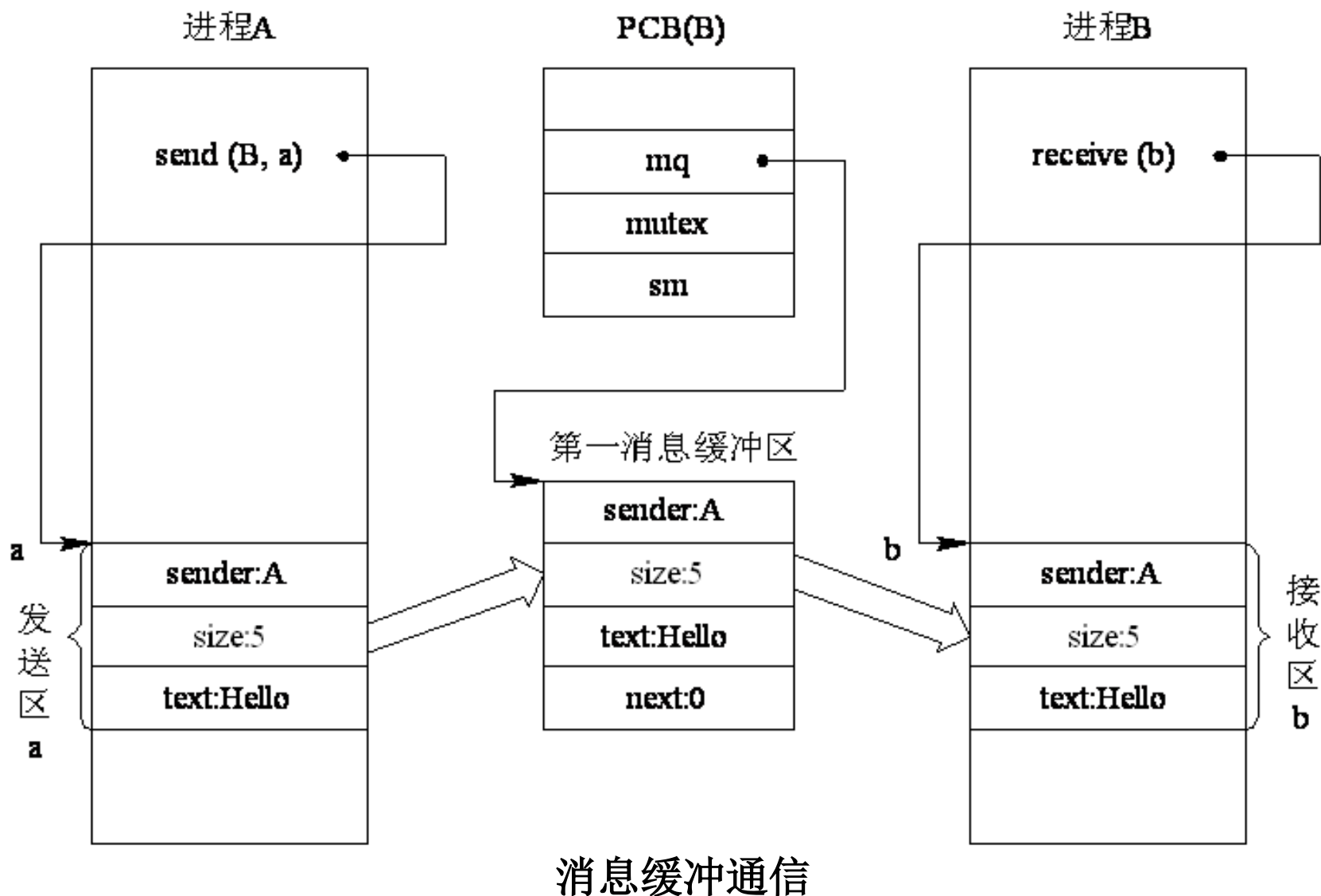
■ 发送原语

1. 根据发送区**a**中所设置的消息长度**a.size**来申请一缓冲区**i**,
2. 把发送区**a**中的信息复制到缓冲区**i**中。
3. 获得接收进程的內部标识符**j**, 然后将**i**挂在**j**的消息缓冲队列**j.mq**上。
4. **j.mq**属于临界资源, 执行**insert**操作的前后需要上锁开锁

■ 接收原语

1. 从自己的消息缓冲队列**j.mq**上摘下第一个消息缓冲区**i**
2. 将数据复制到指定的消息接收区

2.6 进程通信



2.6 进程通信

■ 发送原语程序

```
procedure send(receiver, a)
```

```
begin
```

```
  getbuf(a.size,i);
```

根据**a.size**申请缓冲区；

```
  i.sender: = a.sender; 将发送区a中的信息复制到消息  
  缓冲区之中；
```

```
  i.size: = a.size;
```

```
  i.text: = a.text;
```

```
  i.next: = 0;
```

```
  getid(PCB set, receiver.j); 获得接收进程内部标识符；
```

```
  wait(j.mutex);
```

```
  insert(j.mq, i); 将消息缓冲区插入消息队列； ·
```

```
  signal(j.mutex);
```

```
  signal(j.sm);
```

```
end
```

2.6 进程通信

■ 接收原语程序

procedure receive(b) //参数b是指定消息接收区

begin

j : = internal name; j为接收进程内部的标识符; •

wait(j.sm);

wait(j.mutex);

remove(j.mq, i); 将消息队列中第一个消息移出; •

signal(j.mutex);

b.sender : = i.sender; 将消息缓冲区i中的信息复制到接收区b;

b.size : = i.size;

b.text : = i.text;

end

2.7 线程

■ 进程的不足

□ 为使程序能并发执行，系统必须进行以下一系列操作：

1) 创建进程； 2) 撤消进程； 3) 进程切换

□ 进程同时作为资源分配单位与调度分派单位，限制了并发程度的进一步提高。

■ 传统进程设计的OS不适合SMP结构的计算机。

□ 进程“太重”

□ 分割进程功能，引入线程

2.7 线程

■ 线程

- 每个进程包含多个线程
- 线程作为利用**CPU**的基本单位，是花费开销最小的实体

■ 线程的特点

- 轻型实体。
- 独立调度和分派的基本单位。
- 可并发执行。
- 共享进程资源

■ 线程的**TCB**、参数、状态、控制

2.7 线程

■ 进程与线程

- 在多线程**OS**中，进程是作为拥有系统资源的基本单位，通常的进程都包含多个线程并为它们提供资源，但此时的进程就不再作为一个执行的实体。
- 多线程**OS**中的进程有以下属性：
 - (1) 作为系统资源分配的单位，包括一系列资源。
 - (2) 可包括多个线程。线程属于某个特定进程
 - (3) 进程不是一个可执行的实体。 进程的执行状态实质是它的某个线程执行。

2.7 线程

- 在Windows中的进程与线程
 - 每个windows进程开始于它的被默认创建的第一个线程, 通常称其为主线程。
 - 每个线程都可以创建新的线程
 - 从线程的角度看, 进程表现的更像一个容器:
 - 它代表线程接受分配到的资源(除处理机资源)
 - 为线程提供主体(执行代码+数据)
 - 进程自己没有运行的概念
 - 进程是静态的实体